Budget and Deadline Aware e-Science Workflow Scheduling in Clouds

Vahid Arabnejad, Kris Bubendorfer, and Bryan Ng

Abstract—Basic science is becoming ever more computationally intensive, increasing the need for large-scale compute and storage resources, be they within a High Performance Computer cluster, or more recently within the cloud. In most cases, large scale scientific computation is represented as a workflow for scheduling and runtime provisioning. Such scheduling become an even more challenging problem on cloud systems due to the dynamic nature of the cloud, in particular, the elasticity, the pricing models (both static and dynamic), the non-homogeneous resource types, the vast array of services, and virtualization. This mapping of workflow tasks on to a set of provisioned instances is an example of the general scheduling problem and is NP-complete. In addition, we also need to ensure that certain runtime constraints are met - the most typical being the cost of the computation and the time which that computation requires to complete. In this article, we introduce a new heuristic scheduling algorithm, Budget Deadline Aware Scheduling (BDAS), that addresses eScience workflow scheduling under budget and deadline constraints in Infrastructure as a Service (IaaS) clouds. The novelty of our work is satisfying both budget and deadline constraints while introducing a tunable cost-time trade off over heterogeneous instances. In addition, we study the stability and robustness of our algorithm by performing sensitivity analysis. The results demonstrate that overall BDAS finds a viable schedule for more than 40000 test cases accomplishing both defined constraints: budget and deadline. Moreover, our algorithm achieves a 17.0-23.8 percent higher success rate when compared to state of the art algorithms.

Index Terms—Scientific workflow, scheduling, budget, deadline, cloud

1 Introduction

Scientific discovery is in the midst of a disruptive technological change, where experimental and observational research is being transformed by computational and data-intensive approaches [1]. Researchers in almost every discipline now face new opportunities [2] and challenges that impact every stage of the research lifecycle due to ever growing data and increased analytical complexity [3]. While much of this has in the past utilised dedicated High Performance Computing (HPC) systems, there is an ongoing migration of scientific computing into the various commercial clouds for a number of compelling reasons: elastic clouds offer a variety of accessible and cost-effective computing platforms [4], [5], [6], [7], the on demand model better fits the typically sporadic demands of researchers [8], and finally, rather than resources being used to purchase and maintain dedicated HPC equipment, they are instead used for pay-per-use computation and storage resources offered by cloud vendors.

The cloud presents an opportunity to accelerate scientific discovery by automating [9] computation in workflows, permitting vast numbers of complex compute and data intensive experiments to be executed. A major challenge of the cloud paradigm for e-Science lies in limiting or minimising [10] costs while maintaining or even accelerating throughput. In fact, scheduling workflows and provisioning cloud resources naïvely can have a significant financial penalty—especially in dynamic markets such as the Amazon spot market [11]. The majority of research into scheduling and provisioning has focused on one of, cost or time, as the fundamental workflow scheduling problem is *NP*-complete and optimising multiple constraints, such as cost and time, over a non-uniform set of unlimited resources is nontrivial. Indeed, this complexity leads to long computation times in order to create a reasonable schedule—hence we advocate that a heuristic scheduling approach is needed.

To address this set of problems, we present a new heuristic scheduling algorithm—Budget and Deadline Aware Scheduling for scheduling workflows constrained by both budget and deadline. The BDAS algorithm uses a novel tradeoff factor between time and cost to determine the most viable schedule, and uses this to determine the most appropriate type of instance to provision. To properly evaluate our BDAS algorithm we compare its performance against several state-of-the-art scheduling algorithms over large (1000 task) workflows and evaluate their performance using CloudSim [12]. We also consider a number of metrics and perform a sensitivity analysis of user defined tradeoff priorities to evaluate the stability of the BDAS algorithm.

The remainder of this article is organized as follows: Section 2 discusses the importance of budget and deadline for e-Science in the cloud, while Section 3 gives an overview of existing approaches to scheduling workflows. Section 4 sets out the basic elements of the scheduling problem, while Section 5 introduces the BDAS algorithm. Section 6 details the

• *The authors are with the School of Engineering and Computer Science, Victoria University of Wellington, Wellington 6012, New Zealand. E-mail: {vahid.arabnejad, kris, Bryan.Ng}@ecs.vuw.ac.nz.*

Manuscript received 29 June 2017; revised 13 May 2018; accepted 28 May 2018. Date of publication 29 June 2018; date of current version 12 Dec. 2018. (Corresponding author: Vahid Arabnejad.)
Recommended for acceptance by E. Deelman.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TPDS.2018.2849396

1045-9219 © 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

basis for the evaluation and defines the metrics used in this work. In Section 7 we present the experimental results for BDAS and three other scheduling algorithms over five representative scientific workflows. Finally, we conclude our work in Section 8.

2 Budget & Deadline as Key Drivers for Scheduling

Workflows are scheduled on the basis of execution dependencies, expected execution times, available resources, and typically under either a budget or deadline constraint. In this paper all algorithms are evaluated using criteria based on time and cost efficiency, as well as success rate. Success rate is an oft-neglected metric in scheduling performance, and relates to the ability of a scheduler to build an executable schedule. This section discusses the siginficance of the budget and time constraints with examples.

2.1 Budget

Cost plays an important role in the cloud environment due to its impact on the budget of users and the profitability of providers. Unlike Grid or in-house HPC systems, most cloud providers, such as Amazon, charge users for a fixed interval from the time of provisioning, that is, users pay for the whole interval even if the instance is only used for a fraction of that period. From a users perspective, finding a schedule with a lower makespan for a given budget is the main concern, while from the providers perspective, maximizing utilization is the main concern. Launching too many instances does not lower makespan. Instead, it causes high scheduling overhead and low instance utilization. As one would expect, poor resource provisioning wastes available user budget.

2.2 Deadline

There are many complex and large scale scientific processes in different domains such as weather forecasting, climate modeling, medical modeling and disaster recovery simulation [13] that are examples of deadline-sensitive applications. Some need to be executed quickly or in a specified time. Deadline constrained applications are divided into two types:

- Hard deadline: If deadline is missed, it could have disastrous results on life or environment.
- Soft deadline: The application can tolerate small margins in deadline violation.

The landfall of a storm can be predicted by coastal hazard applications and used to determine whether to send out evacuation orders. In such workflows, missing the execution deadline is considered a failure because safety and lives are at risk [14]. Weather forecast workflows [15] that forecast the weather for the next day (or week) and medical applications that analyze data related to patients activity are examples of applications with soft deadline.

3 Related Work

Scheduling workflow tasks to resources while meeting workflow dependencies and constraints is known as the Workflow Scheduling Problem (WSP)–a class of problem that is known to be NP-complete [16]. Allocating workflow tasks to resources can be separated into two stages, the first being scheduling and the second is provisioning [17].

Given a set of resources, the workflow task scheduling phase aims to determine the optimal execution order and task placement with respect to user and workflow constraints [18], [19]. The resource provisioning phase aims to determine the number and type of resources required and then to reserve these resources for workflow execution [20], [21]. While the majority of cloud scheduling systems necessarily include both scheduling and provisioning stages, prior research tends to focus on the scheduling phase, under the assumption that a pre-identified pool of (often homogenous) resources are used for execution, and with the goal of optimizing workflow execution time (makespan) without considering resource cost. In a commercial cloud environment this set of assumptions no longer holds.

Based on the user-defined constraints for the budget and deadline, a combination of different solutions are proposed in [22] to customers. Their main objective is to produce a set of solutions from which a user makes a selection. However, their approach is designed for users in Grid environment. Pricing schemes in cloud systems are based on lease intervals from the time of provisioning, even if the instance is only used for a fraction of that period. However, cost in Grid is calculated based on the accumulated cost of requested services.

Meta-heuristics such as Genetic Algorithm(GA), Ant Colony Optimization(ACO) and Particle Swarm Optimization (PSO) are used to tackle the workflow scheduling problem with multiple constraints. In [23], genetic algorithm approach was used to address scheduling optimization problems in workflow applications, based on two QoS constraints, deadline and budget.

In [24], Chen et al. use Ant Colony Optimization to schedule the large-scale workflows with three QoS parameters, namely, cost, time and reliability. Users can specify two of the constraints and ACO can find an optimized answer for the third constraint. A Bi-Criteria Priority based Particle Swarm Optimization (BPSO) was proposed in [25] to solve workflow scheduling problem in cloud under given the deadline and budget constraints. BPSO was shown to reduce the execution cost by up to 50 percent compared to several then state-of-the-art algorithms under the same Deadline and Budget Constraint.

While search-based and meta-heuristic strategies produce acceptable answers, they are usually time-consuming algorithms based on their need for an initialization phase to reach the final answer. Another important point that needs to be considered in these algorithms is that lack of ability to perform well when some changes happen during execution. Moreover, they do not respond well in dynamic scheduling because of the required dynamic phase.

There are few heuristic based studies addressing workflow scheduling and resource provisioning while considering both deadline and budget constraints at the same time in cloud. In [26] compromised cost-time algorithm is presented that deadline and budget are considered as QoS parameters. Their approach is different from ours in two ways. First, the main step of [26] is rescheduling tasks that causes higher time complexity. Second, they did not consider heterogeneous instances in the problem formulation.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

In [27], Zheng and Sakellariou, introduce an extended version of HEFT algorithm with both budget and deadline constraints (BHEFT). BHEFT checks if a workflow can be scheduled based on the available budget and deadlines. To select the best possible instance in BHEFT, two variables named Spare Application Budget and Current Task Budget are used. This work while developed for cloud, uses a grid cost model where the number of resources is fixed. Moreover, in their presented algorithms, there is no mechanism to control the deadline if in any steps the algorithm exceeds the user-defined deadline. Recently, an extension of BHEFT is presented in [28] under deadline and budget constraints in cloud environment.

In [29], Poola et al. present a robust scheduling algorithm for heterogeneous cloud resources with time and cost constraints. Tasks are categorized based on partial critical paths, in a similar way to IC-PCP [30]. Three resource selection policies are used: deadline, cost and robustness, and each user can prioritize the policies independently. The scheduling algorithms proposed by Malawski et al. in [31] aim to maximize the number of serviced workflows while meeting given budget and deadline constraints. These scheduling algorithms are designed for workflows in an Infrastructure as a Service (IaaS) cloud. However, the authors consider only one instance type rather than the multiple instance types that are currently supported by commercial providers.

Most of the workflow scheduling research efforts over the past years have focused on the scheduling of the workflows under a single constraint. The review of the above mentioned papers reveal a need for an algorithm that considers both budget and deadline constraints at the same time in cloud environment. From our survey of the related work, the algorithms in [29] and [28] were chosen in order to benchmark the performance of our Budget Deadline Aware Scheduling algorithm. These works are recent and consider cost and time as constraints rather than optimizing them.

4 Problem Definition

4.1 Application Model

Workflows are the most widely used models for representing and managing complex distributed scientific computations [13]. A Directed Acyclic Graph (DAG) is the most common abstraction of a workflow. Using a DAG abstraction, a workflow is defined as a graph $G = (T, E)$ where $T = \{t_0, t_1, ..., t_n\}$ is a set of tasks represented by vertices and $E = \{e_{i,j} \mid t_i, t_j \in T\}$ is a set of directed edges denoting data or control dependencies between tasks. An edge $e_{i,j} \in E$ represents the precedence constraint as a directed arc between two tasks t_i and t_j where $t_i, t_j \in T$. The edge indicates that task t_j can start only after completing the execution of task t_i with all data received from t_i and this implies that task t_i is the parent of task t_j, and task t_j is the successor or child of task t_i. Each task may have one or more parents or children. Task t_i cannot start until all parents have completed.

4.2 System Model

In this article, we adopt the IaaS service model. The IaaS paradigm provides a service by offering instance types containing various amounts of CPU, memory, storage and network bandwidth at different prices. Workflows are executed on different instance types, and each instance type is associated with a set of resources.

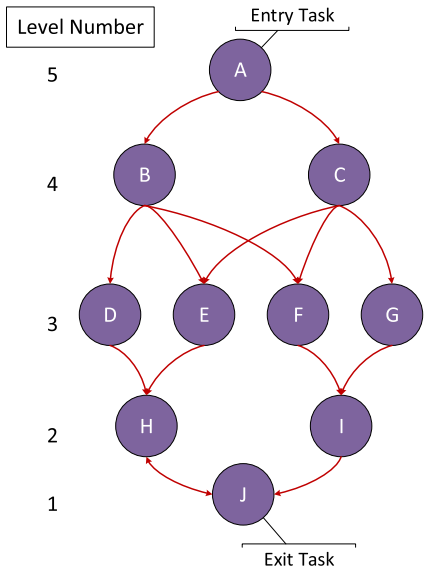


Fig. 1. A sample DAG with ten tasks and five levels.

We use a resource model based on the Amazon Elastic Compute cloud, where instances are provisioned on demand. The pricing model is a pay as you go with minimum hourly billing. Under this pricing model, if an instance is used for one minute, a user has to pay for the whole hour. We assume that cloud vendors provide access to unlimited number of instances and the instances are heterogeneous (denoted by $P = \{p_0, p_1 \ldots p_h\}$, where h is the index of the instance type). We also assume that all instances and storage services are located in the same region and also assume that the average bandwidth between the instances is essentially identical.

4.3 Definitions

Most studies on workflow scheduling assume that estimated execution time for workflow tasks are known. Task runtimes can be estimated through analytical modeling, empirical modeling and historical data. In this article, we use scientific workflows generated using trace data from real applications [32]. Execution time (computation cost) for task t_i on instance p_j is denoted by $w_{t_i}^{p_j}$.

All immediate predecessors of task t_i are defined as:

$$pred(t_i) = \{t_j \mid (t_j, t_i) \in T\}. \tag{1}$$

Similarly, all immediate successors of task t_i are defined as:

$$succ(t_i) = \{t_j \mid (t_i, t_j) \in T\}. \tag{2}$$

For example in Fig. 1, predecessors of task H are D and E, successors of C are E, F and G. A task without any parent is an entry task and a task without any children is called an exit task. In Fig. 1, task A is an entry task and task J is an exit task. Thus, by definition we have:

$$pred(t_{entry}) = \{\emptyset\}, \tag{3}$$

$$succ(t_{exit}) = \{\emptyset\}. \tag{4}$$

The completion time of a workflow is called the schedule length or makespan (denoted by $\mathcal{L}_{ms}$). Because t_{exit} is the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Five Main Phases of BDAS

	Phase	Description
5.1	Workflow partitioning	The workflow is partitioned into dependency free bags of tasks, called levels.
5.2	Budget Distribution	The user-defined budget (B) is then allocated to each defined level using "All in" strategy.
5.3	Deadline Distribution	The user-defined deadline (D) is divided and distributed between levels. Each level gets its own level deadline. All tasks in the same level, have the same level-deadline.
5.4	Task Selection	A task is selected based on its priority in the ready list for execution.
5.5	Instance Selection	In this phase, we introduce new trade-off between Cost and Time. We find the best combination of cost and time to select the most suitable instance.

last task that can be executed, the time until completing the exit task is defined as the makespan of a workflow. Thus,

$$\mathcal{L}_{ms} = FT(t_{exit}), \tag{5}$$

where $FT(t_{exit})$ is the finish time of the last task in a workflow.

The amount of data transferred from task t_i to task t_j is called communication time (denoted by $C_{i,j}$), and this time is calculated as:

$$C_{i,j} = \begin{cases} \frac{data}{\beta}, & p_i \neq p_j \\ 0, & p_i = p_j. \end{cases} \tag{6}$$

If task t_i and task t_j are executed on the same instance (denoted by $p_i = p_j$), data transfer between them is local and the communication cost is defined as zero. Otherwise, the communication cost is the ratio between the size of data ($data$) to be transferred from task t_i to t_j to the average connection bandwidth (β) within a data-center.

The Earliest Start Time (EST) of a task t_i is calculated on the instance with the shortest execution time and defined as:

$$EST(i) = \begin{cases} 0, & t_i = t_{entry} \\ \max_{t_j \in pred(t_i)}\{EST(t_j) + w_{t_j} + C_{i,j}\}, & \text{otherwise}, \end{cases} \tag{7}$$

where w_{t_j} is the execution time of task t_j on the fastest instance type. The cost of executing task t_i on instance p_j is calculated as:

$$TaskCost_{t_i}^{p_j} = \left\lceil \frac{w_{t_i}^{p_j}}{N_t} \right\rceil * c_j, \tag{8}$$

where c_j is the cost of instance p_j for one time interval and N_t is the time of an interval. Finally, the overall cost of executing all tasks in a workflow is defined as:

$$Cost_o = \sum_{t_i \in G} TaskCost_{t_i}^{p_j}. \tag{9}$$

where G is the DAG (see sub-section 4.1).

5 The BDAS Algorithm

In this section, we describe our Budget-Deadline Aware Scheduling algorithm that attempts to meet budget and deadline constraints. The BDAS algorithm is divided into five main phases that is indexed in Table 1.

5.1 Workflow Partitioning

We aim to maximize task parallelism by partitioning tasks such that there are no dependencies between tasks in each level. Each level can therefore be thought of as a bag of tasks (BoT) containing a set of independent tasks.

In this step, the tasks are partitioned into different levels based on their respective dependencies. Subsequently, the user-defined deadline (D) and user-defined budget (B) are distributed over the levels established in the preprocessing step. Each level gets its own level deadline and budget and all tasks in the same level have the same level-deadline and level-budget.

There are two main algorithms for allocating tasks into different levels, Deadline Bottom Level (DBL) [33] and Deadline Top Level (DTL) [34]. Both DBL and DBT algorithms categorize tasks in bottom-top direction and top-bottom direction, respectively. In this article, we use the DBL algorithm to partition tasks over the different levels.

We describe the level of task t_i as an integer representing the maximum number of edges in the paths from task t_i to the exit task (see Fig. 1). The level number (denoted by L) associates a task to a BoT. For the exit task, the level number is always 1, and for the other tasks, it is determined by:

$$L(t_i) = \max_{t_j \in succ(t_i)} \{L(t_j) + 1\}, \tag{10}$$

where $succ(t_i)$ denotes the set of immediate successors of task t_i. All tasks are then grouped into Task Level Sets (TLS) based on their levels:

$$\text{TLS}(\ell) = \{t_i | L(t_i) = \ell\}, \tag{11}$$

where ℓ is an integer denoting the level in $[1 \ldots L(t_{entry})]$.

5.2 The "All in" Budget Distribution

The financial cost and total execution time of a workflow depends on the number and types of instances requested during resource provisioning. The cost plays a significant role in a cloud environment as users wish to minimise costs and providers maximise profits. Most cloud providers, for example Amazon, charge users for a minimum period of time—even if the instance is only used for a fraction of the period leased.

The idea of budget distribution is to distribute a global budget (B) among different levels and schedule each task on an instance considering the available sub-budget $(subB(\ell))$ assigned to a level. Essentially, we transform the workflow into internally dependency free "bag of tasks" and we then distribute the workflow budget over these levels. In our previous work [35], we presented new notions for distributing budget based on the dependency structure inherent in workflows levels. In addition we proposed several new strategies for sharing or distributing budget over these levels.

The most significant differences in each strategy lie in the calculation of the sub-budget. In some strategies, we have a Budget Factor (BF) that determines a share of the budget for each level. Each sub-budget assigned to a level is termed the level budget. Overall, the strategy that performed the best for all workflows in terms of both success rate and

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

makespan was "All in". The "All in" strategy assigns the budget to the earliest levels and was shown to be beneficial in terms of reducing makespan. An important concept in our algorithm is trickling down unused budget to later levels and this is expressed by Equation (12):

$$Spare_B = subB(\ell) - \sum_{t_i \in \mathrm{TLS}(\ell)} TaskCost_{t_i}, \tag{12}$$

where $TaskCost_{t_i}$ is defined in 8. We define Spare Budget as the amount of money remains after allocating all tasks in the level ℓ. We then add the leftover to the next level $(\ell + 1)$. The "All in" gave the best success rates—and the best performance for data intensive workflows due to the resulting data locality, from fewer but more powerful instances.

5.3 Deadline Distribution

One critical element is the provisioning of pay-per-use instances, and the subsequent scheduling of workflow tasks over them—in essence we need to complete execution on time. One approach is to divide or distribute the deadline over the workflow as sub-deadlines to ensure a more manageable constraint satisfaction problem, and then to provision the instances to meet these sub-deadlines. The main idea of deadline distribution is to distribute deadline among different levels and complete its execution before any assigned sub-deadline such that the global deadline can be met.

In our previous work [36], we explored a range of strategies for distributing a deadline over a workflow. We found that the choice of distribution strategy has a significant impact on performance. Some strategies exhibit performance that is strongly dependent on the workflow size and structure including process, pipeline, data distribution, data aggregation and data redistribution. In general, the strategy which takes into consideration the execution time of each level as well as number of tasks in the level, yields the lowest cost.

Once all tasks are assigned to their respective levels, the tasks are proportionally distributed across each level based on the user deadline (D). Each sub-deadline assigned to a level is termed the level deadline $(subD(\ell))$. To meet the overall deadline, we attempt to ensure that every task in a level can complete its execution before the assigned sub-deadline. First, the initial estimated deadline for each level (ℓ) is calculated as:

$$subD(\ell) = \max_{t_i \in \mathrm{TLS}(\ell)} \{\mathrm{ECT}(t_i)\}, \tag{13}$$

where $\mathrm{ECT}(t_i)$ denotes the Earliest Completion Time (ECT) of task t_i over all instances and the ECT is defined as

$$\mathrm{ECT}(t_i) = \max_{\ell \in pred(t_i)} \{subD(\ell), EST(t_i)\} + w_{t_i}, \tag{14}$$

where $EST(t_i)$ is defined in Equation (7), $pred(t_i)$ denotes the set of predecessors of task t_i; w_{t_i} denotes the minimum execution duration for task t_i and ℓ indicates the parent level t_i. The task, t_{entry} has no predecessors, its ECT is equal to zero. In Equation (13), the maximum ECT of all tasks in a level is used as the overall estimate for that level. This duration is effectively the absolute minimum time that is required for all tasks in a level to complete execution in parallel.

After calculating the estimated deadline value for all levels, we distribute the user deadline among all tasks non-uniformly based on a deadline proportion and the number of tasks in each level. For this purpose, we first introduce the proportion unit denoted by $\propto_{deadline}$ in Equation (15):

$$\propto_{deadline} = \sum_{\ell=1}^{t_{entry}} |subD(\ell)| \times |\mathrm{TLS}(\ell)|, \tag{15}$$

where $|subD(\ell)|$ is the length of each level deadline calculated by the differentiate of level ℓ and $\ell - 1$. Also, $|\mathrm{TLS}(\ell)|$ is the number of tasks on each level.

Now we define the Deadline Factor (DF) as:

$$DF = \frac{D - subD(1)}{\propto_{deadline}}, \tag{16}$$

where $subD(1)$ is the level that contains the exit task. The numerator in Equation (16) is the left over deadline subtracting the total deadline (D) and the soonest possible finish time for a workflow($subD(1)$).

We then update the length of each level deadline as a function of this Deadline Factor to each level as follows:

$$subD(\ell) = DF \times |subD(\ell)| \times |\mathrm{TLS}(\ell)| + subD(\ell). \tag{17}$$

Intuitively, the levels with longer executing tasks and higher number of tasks, gain a larger share of the user deadline.

5.4 Task Selection

There are no dependencies between tasks that occupy the same level and are therefore executable in parallel and placed simultaneously on the read list. Tasks are selected from the ready list using Earliest Start Time.

In BDAS, tasks are executed level by level which means tasks may only start execution once all tasks in the prior level have been scheduled. This appears at first to be a potential limitation in the performance of the BDAS algorithm—as it could be the case that a ready task for which all dependencies in the previous level have finished execution is forced to wait on non-dependencies. However this is required by the budget trickling mechanism—essentially, we need to have resolved the budget spend by one layer before we can fully determine the budget for the next.

In addition, due to the use of Earliest Start Time, the overall number of temporarily delayed tasks is small and extremely unlikely to be a significant factor in the overall scheduling performance of the algorithm. With EST, even if a task in a later level executes prior to the completion of an earlier adjacent level, the properties of EST mean that any subsequent levels do not overlap in terms of EST and therefore would not execute anyway. On testing, we found that the number of delayed tasks in workflows of 1000 tasks were at worst, three tasks, when we removed the level limitation imposed by BDAS. This would not be the case if we were using an alternative task selection algorithm such as min-exe-time.

5.5 Instance Selection

At the point the algorithms perform instance selection: (i) each task is already assigned a level, (ii) the budget and deadline for each level is already determined, and (iii) the priority of each ready task is already assigned. During

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Different Possible Ranges for Cost and Time

Cost	Time	Description
> 0	> 0	In the best scenario, we get enough budget from a user to schedule a workflow. Also, the deadline is relaxed and there is no pressure on finishing the scheduling. In this situations, while distributing budget and deadline, each level can get a big share of them. Therefore, both values of Cost and Time are positive as task levels sub-budget and task levels sub- deadline are big enough. Moreover, at the beginning of the scheduling process when no budget has spent or there is enough time for scheduling the current task t_i on any instances, both values are positive.
≤ 0	> 0	The total assigned budget for the level ℓ has already been spent ($subB^{\ell_{t_i}} = 0$). Therefore, we cannot launch a new instance as there is no budget left. Another probable situation is the cost of current task t_i on the instance p_j is higher that the leftover sub-Budget (in the numerator of equation (19)). If either of these conditions is true, equation (19) less than or equal zero.
> 0	≤ 0	In tight deadlines, there is a possibility that cheaper instances cant meet the task levels sub-deadline. Therefore, the value of ECT in equation (18) is greater or equal to task levels sub-deadline. If this condition is met, the value of equation (18) becomes negative or zero.
≤ 0	≤ 0	In tight budget and deadline, where current sub-budget and sub-deadline are smaller than values of $TaskCost$ and ECT, respectively, for task t_i on an instance, both equations (19) and (18) can be negative or zero.

instance selection, a trade off must be made between execution time and cost. To demonstrate this trade off, we show the expressions for both the time and the cost of executing each task on each instance type in Equations (18) and (19), forming a pair of expressions for Time and Cost.

First, the time needed for the current task, t_i, on the instance p_j is denoted by $\mathrm{ECT}(t_i|p_j)$. The ECT is the earliest time that a task can finish on an instance which is defined earlier in Equation (14). Using this observation, we can then compute how much the estimated level deadline of the current task differs from the earliest completion time of task on the instance p_j:

$$\mathrm{Time}_{t_i}^{p_j} = \frac{subD^{\ell_{t_i}} - \mathrm{ECT}(t_i|p_j)}{subD^{\ell_{t_i}} - \mathrm{ECT}(min)}. \tag{18}$$

In Equation (18), $subD^{\ell_{t_i}}$ is the deadline that is assigned to the level which contains the task t_i. Also, $\mathrm{ECT}(min)$ is the minimum execution time among all instances that keeps our current task on schedule.

The values of Time for task t_i are related to instance types, wherein a lower value of Time means running on a cheaper instance. The reason is that the values of $\mathrm{ECT}(t_i|p_j)$ is bigger on an instance with a lower processing capacity. Also, if the value of Time is negative, it means that the current task on the selected instance will exceed the level deadline i.e., $\mathrm{ECT}(t_i, p_j) > Level_{deadline}^{\ell_{t_i}}$.

In the expression for $TaskCost$, given earlier in Equation (8), $TaskCost_{t_i}^{p_j}$ refers to the cost of scheduling the current task t_i on instance p_j. The cost of executing task t_i among all instances p_j is defined as:

$$\mathrm{Cost}_{t_i}^{p_j} = \frac{subB^{\ell_{t_i}} - TaskCost_{t_i}^{p_j}}{subB^{\ell_{t_i}} - TaskCost_{\min}}, \tag{19}$$

where $subB^{\ell_{t_i}}$ denotes the budget assigned to the level which contains the current task. The best cost (minimum cost) of executing the task t_i among all instances is $TaskCost_{\min}$.

When an instance is first provisioned, the instance is billed on an hourly interval until it is terminated. Therefore, the first task assigned to an instance in a particular billing interval incurs the entire cost of that interval. As a consequence, if other tasks can be executed during that paid interval, then there is no additional execution cost for executing them. Ideally, during instance selection, the reuse of such instances lead to lowering the execution cost. This situation comes true when Cost in Equation (19) is 1 (The value of $TaskCost_{t_i}^{p_j}$ and $TaskCost_{\min}$ are zero).

To find the best instance, we use a Cost Time Trade-off Factor (CTTF) in Equation (20) that considers a trade-off between cost and time. The CTTF value is trying to adjust the cost and time for current task among all instances. For cost minimization purpose, most of the proposed algorithms like [31] schedule tasks on the cheapest available instance (slower) while still meeting its assigned sub-deadline. However, with this strategy, tasks can take much longer time if the resources are slower and leads to some delay in the EST of its children. To avoid this hold up, more expensive instances are better choices to meet deadline, that probably lead to exceeding the budget. Therefore, the key concept of introducing CTTF in Equation (20) is to make a trade-off between time and cost.

While calculating Equations (19) and (18) for a task among all instances, four different combinations of cost and time ranges are possible listed in Table 2. As we mentioned before, a trade off between Cost and Time should be considered to find the best instance for each task.

The trade-off function is calculated as follows:

$$\mathrm{CTTF}_{t_i}^{p_j} = \mathrm{Cost}_{t_i}^{p_j} + \mathrm{Time}_{t_i}^{p_j}. \tag{20}$$

The instance with highest CTTF value is selected for execution the task t_i.

6 Evaluation

In this section we compare the performance of the BDAS algorithm, with recent published papers in [29] and [28] that match our goal and conditions. We use a simulator to compare the performance of all four algorithms. Simulations are well accepted as the first approach to evaluate new techniques for workflow scheduling problem. It allows researchers

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Instance Types Based on Amazon EC2

Type	ECU	Memory(GB)	Cost($)
m3.medium	3	3.75	0.067
m4.large	6.5	8	0.126
m3.xlarge	13	15	0.266
m4.2xlarge	26	32	0.504
m4.4xlarge	53.5	64	1.008
m4.10xlarge	124.5	160	2.520

to test the performance of newly developed algorithms under a controlled setting. For this purpose, all four algorithms are implemented and evaluated in CloudSim [12].

Our simulation scenario is configured as a single datacenter and six different instance types. The characteristics of the instances are based on the US-east Amazon region[1] presented in Table 3, and were accurate in March 2016. The average bandwidth between instances is fixed to 20 MBps [37]. The processing capacity of an EC2 unit is estimated at one Million Floating Point Operations Per Second (MFLOPS) [38]. The estimated execution times are scaled by instance type CPU performance, that means 1 second of each task in a workflow runs for 1 second on an instance with one ECU. This is a fairly common simplifying assumption using the theoretical maximum based on the results from Ostermann [38] which has been used frequently in related work such as [39] and [40].

In an ideal cloud environment, there is no provisioning delay in resource allocation. However, some factors such as the time of day, operating system, instance type, location of the data-center, and requested resources can cause delays in startup time [41]. Therefore, in our simulation, we adopted a 97-second boot time based on previous measurements of EC2 [41].

In order to evaluate the performance of our algorithms with a realistic load, we use five common scientific workflows: Cybershake, Epigenomics, Montage, LIGO and SIPHT. The characteristics and task composition of these workflows have been analyzed in published works cited in the related work section [32], [42]. To evaluate the performance of these algorithms, we choose different deadlines chosen from tight to relaxed. For this purpose, we calculate the fastest schedule (denoted by FS) as a baseline schedule. Effectively, this baseline is the fastest possible execution—ignoring costs. The fastest schedule is expressed as:

$$FS = \sum_{t_i \in CP} (w_i^j), \tag{21}$$

where w_i^j is the computation cost of task t_i on the fastest instance p_j. Additionally, different budget ranges are considered for the scientific datasets from lowest possible through to sufficient. As a baseline schedule, the lowest budget (denoted by LB) to schedule a workflow is given by equation (22)

$$LB = \sum_{\forall t_i \in G} TaskCost_{t_i}^{p_j}, \tag{22}$$

where p_j is the cheapest instance. To achieve this, all tasks are executed on the instance with the lowest cost (the cheapest instance). This assignment gives us the lowest possible cost required for executing a workflow, irrespective of finishing time.

Using Equations (21) and (22), we calculate a variation for budget and deadline from tight to moderate to relaxed:

$$\text{deadline} = \alpha * FS, \quad 0 < \alpha < 20, \tag{23}$$

$$\text{budget} = \beta * LB, \quad 0 < \beta < 20. \tag{24}$$

Both deadline factor α and budget factor β start from 1 to consider very tight values and are increased by one up to a value of 20, which results in a very relaxed deadline and sufficient budget.

The Amazon EC2 instances charge on an hourly interval from the time of provisioning. We configure our simulator to reflect this charging model and we use a time interval of 60 minutes in our simulations. To compare performance with respect to different workflow sizes we evaluated workflows with 50, 100, 200, 500 and 1000 tasks. However, as these results did not vary significantly we present here only workflows with 1000 tasks.

We used the Pegasus workflow generator [32] (commit 4241001) to create representative workflows with the same structure as five real world scientific workflows (Cybershake, Epigenomics, Montage, LIGO and SIPHT). For each workflow structure, and each deadline factor, 100 distinct Pegasus generated workflows were scheduled in CloudSim and the performance of the scheduling algorithms are detailed in the following section.

6.1 Performance Metrics

To evaluate the algorithms under test, we use the following performance metrics: Success Rate (SR), Cost Ratio (CR) and Time Ratio (TR). Each metric is explained as follows:

- Success Rate (SR): success rate of each algorithm (SR), calculated as the ratio between the number of simulation runs that successfully met the scheduling both constraints, deadline and budget, and the total number of simulation runs (denoted by n_{Tot}), defined as:

$$\text{SR} = \frac{n(k)}{n_{Tot}}, \tag{25}$$

  where $n(k)$ is the cardinality of the set k and $n_{Tot} = 100$.
- Cost Ratio (CR): To compare the achieved cost between each algorithm, we use a Cost Ratio which calculates by dividing overall cost (expressed in equation 9) by given budget (B).

$$\text{CR} = \frac{Cost_o}{B}. \tag{26}$$

  A CR value greater than one indicates a cost larger than the budget which counts as a failure to meet the defined budget. CR value of less than one indicates that the scheduled workflow meets the budget.
- Time Ratio (TR): With the same reasoning with CR, we consider the value of Time Ratio metric while

1. https://aws.amazon.com/ec2/pricing/

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

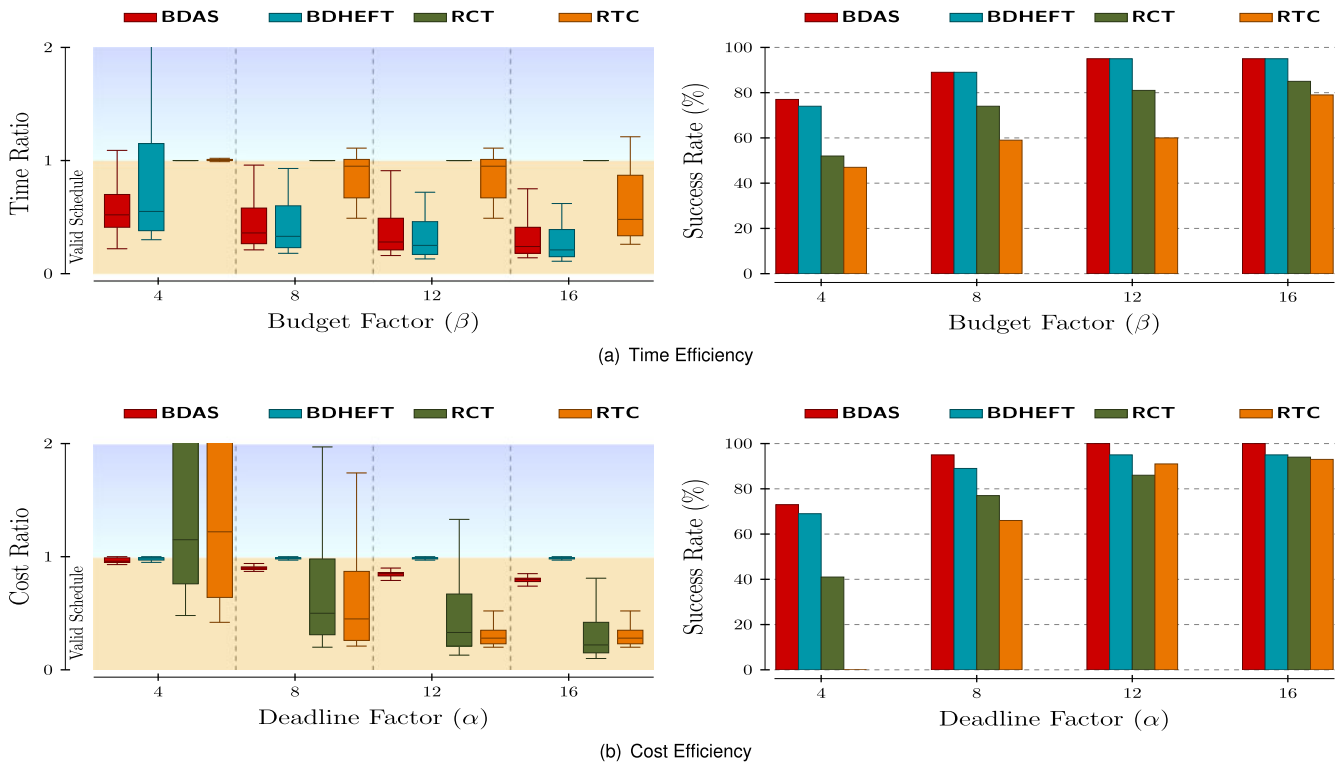


Fig. 2. CYBERSHAKE.

smaller than one means a user defined deadline (D) is met.

$$\mathrm{TR} = \frac{Makespan}{D}. \tag{27}$$

7 Experimental Results

We defined 20 different deadline factors (given by Equation (23)) and 20 different budget factors (given by Equation (24)). Therefore, for our experiments permuting both factors yields 400 different cases per workflow and a grand total of 40000 test cases for all workflows.

Comparative studies were conducted by inspecting box plots for cost ratio and time ratio defined in Equations (26) and (27), respectively. A box plot shows the degree of dispersion of cost ration and time ratio. The lowest "dash" denotes the minimum, while the upper "dash" denotes the maximum. The median is marked in the box, the top boundary and bottom boundary denotes the upper and lower quartile.

To facilitate the presentation, the box plots are divided into two areas named valid schedule and invalid schedule. The valid schedule starts from 0 to 1 indicating an algorithm meets the defined constraint.

We present the results in two dimensions:

(i) Time Efficiency (Budget Factor versus Time Ratio): We chose four different ranges of budget factor in our experiments giving us more detail about the performance of all algorithms to simplify interpretation of results. For each budget factor, we draw a box plot for all achieved makespan (summarized by Time Ratio factor in (27)). With this plot we can analyze the behavior of each algorithm on meeting/failing and utilizing deadlines for a constant budget factor, for example $\beta = 4$. Moreover, corresponding success rates of each budget factor is plotted.

(ii) Cost Efficiency (Deadline Factor versus Cost Ratio): We choose four different ranges of deadline factor, including very tight deadlines ($\alpha = 4$), progressing to more relaxed deadline ($\alpha = 16$). The y-axis in this plot is Cost ratio(Equation (26)) indicating meeting/failing and utilizing all ranges of budget for a given deadline. Also, corresponding success rates of each deadline factor is plotted.

The primary objective for any workflow scheduling algorithm is finding a schedule without violating defined constraints. For a user, shortening the execution time with minimum cost is of interest. Therefore, precise behavior of each algorithm on expenditure budget/deadline is of interest.

7.1 CYBERSHAKE

Fig. 2 shows the results achieved for CYBERSHAKE. For Time efficiency (Fig. 2a), BDAS and BDHEFT have the best performance by achieving a smaller time ratio than others. This means that both algorithms were able to find a schedule ahead of the given deadlines. Moreover, the related success rate shows BDAS and BDHEFT outperform both RCT and RTC.

However, in terms of Cost Efficiency, both BDAS and BDHEFT utilized up to 100 percent of the assigned budget in CYBERSHAKE workflow (Fig. 2b). The RCT and RTC algorithms are better at saving cost compared to BDAS and BDHEFT. The RTC algorithm has the worst success rate among the four, specially in tight deadlines. Indeed RTC experienced a 100 percent failure when deadline factor, α, is 4.

7.2 EPIGENOMICS

The results obtained for the EPIGENOMICS are shown in Fig. 3. The BDAS algorithm is capable of meeting constraints

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

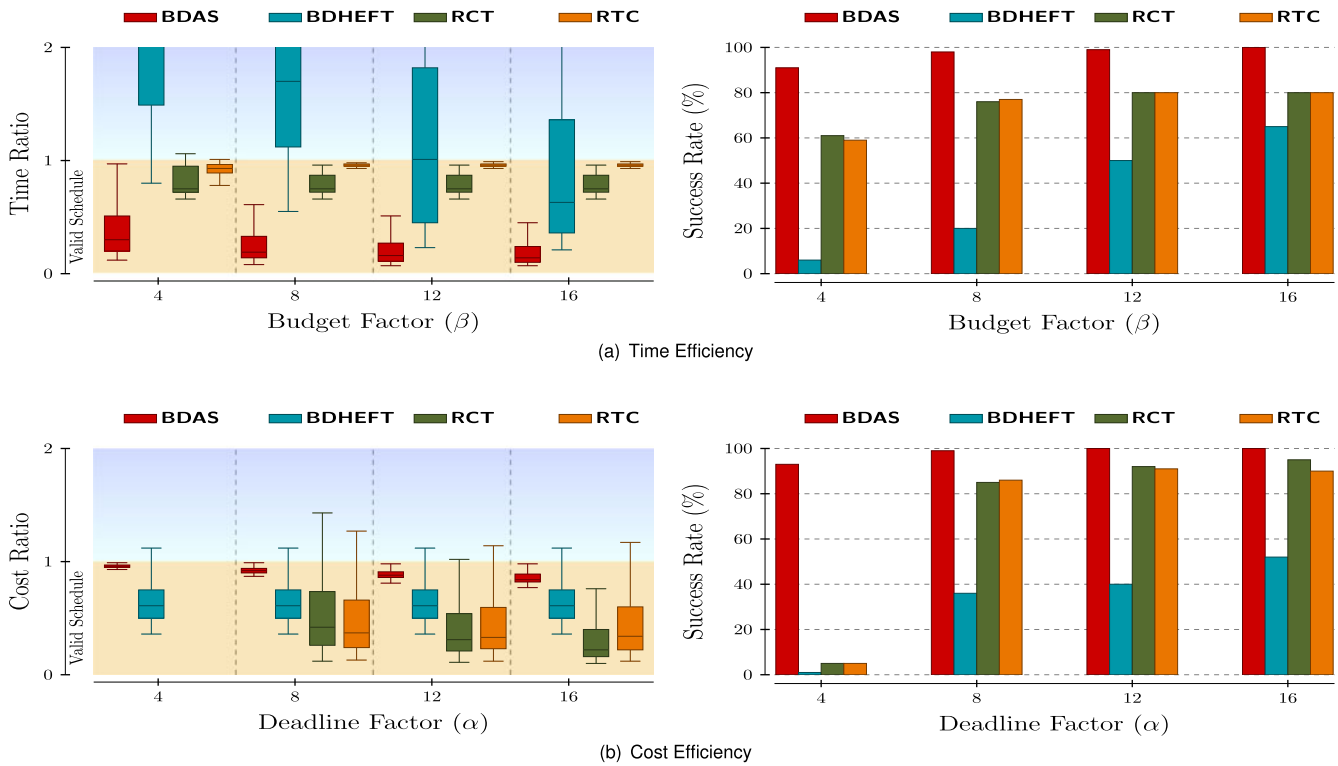


Fig. 3. EPIGENOMICS.

nearly 100 percent. It almost use the whole budget while finding a schedule sooner than defined deadline. The BDAS for EPIGENOMICS, utilizes the whole budget and finishes sooner than defined deadline leading to users satisfaction. The behavior of RCT and RTC are opposite of the BDAS algorithm whereby they utilize deadline rather than budget. An interesting feature of RCT in Fig. 3a, happens when budget factor, β, is 4. As can be seen, almost all time ratio results yield a valid schedule, while achieving success rate of 60 percent. It means, 40 percent of failures in this algorithm happens due to violating budget. The opposite is observed for BDHEFT in 3b, when deadline factor, α, is 8. Nearly all cost ratio values are inside the valid schedule, however the success rate is less than 40 percent. The reason is most of the failures are caused by exceeding the given deadline.

The highest failure occurs in EPIGENOMICS while deadline factor α is 4, BHEFT,RCT and RTC can find a schedule for less than 7 percent before they're constraints are reached. However, our BDAS algorithm achieves more than 90 percent success rate under the same conditions.

7.3 LIGO

Fig. 4 shows the results achieved for LIGO. In general, the BDAS heuristic has the best performance in terms of meeting given constraints, budget and deadline. The behavior of BDAS is similar to EPIGENOMICS, whereby it utilizes the full budget and completes the execution of workflows sooner than prescribed deadline. On the contrary, the RCT and RTC algorithm spend less budget while exhausting the deadline.

The worst performance is observed with BDHEFT in first and second defined deadline factor, α, with almost 100 percent failure. All related cost ratio values for mentioned deadline factors are placed in the valid schedule, as it can be seen in Fig. 4b. All failures are due to violating the prescribed deadline.

7.4 MONTAGE

The results obtained from our simulations for the MONTAGE workflow are shown in Fig. 5. All algorithms exhibit nearly identical success rate for this application in specified intervals. The figures for BDAS in both plots show similar decrease trends in time ratio and cost ratio. In terms of time efficiency (Fig. 5a), the time ratio obtained by BDHEFT is smaller than the ratio obtained by other algorithms in all intervals. The behavior of BDHEFT for different deadline intervals implies that it maintains a constant level of performance by utilizing all the allocated budget.

Apart from the lowest considered budget factor and deadline factor in our experiments, all four algorithms obtained ratios remained under one for MONTAGE (valid schedule). In MONTAGE, all algorithms gained the highest total success rate compared to other scientific workflows as discussed in Section 7.6.

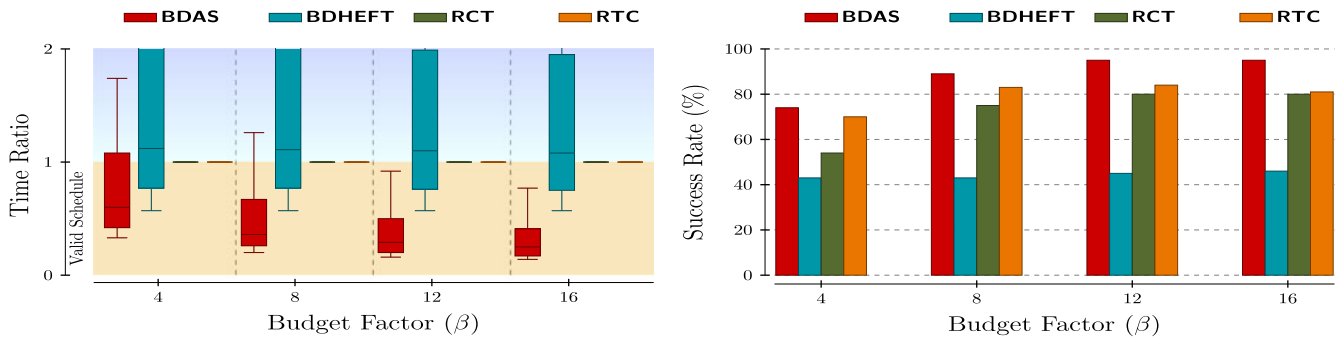
7.5 SIPHT

The time ratio and cost ratio obtained by tested algorithms for SIPHT workflows are shown in Figs. 6a and 6b, respectively.

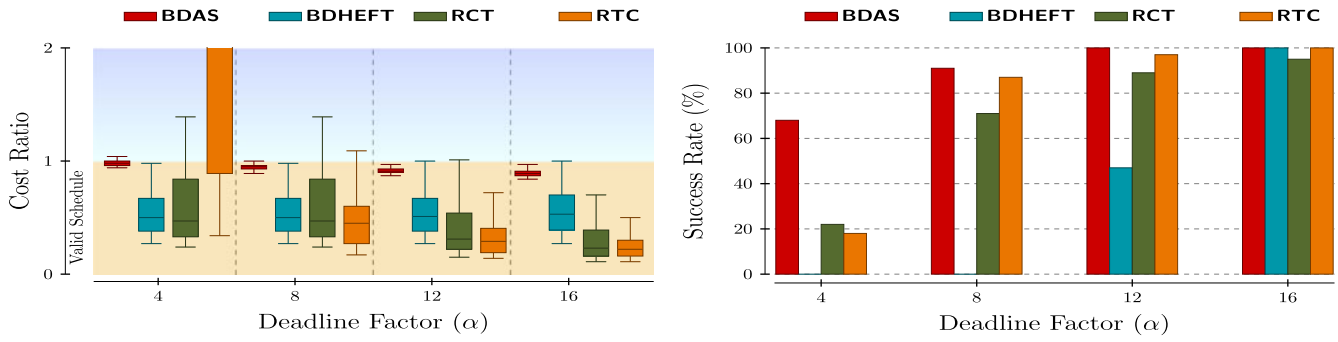
The maximum of box plots for RCT and RTC are not shown because any ratio greater than 1 means a failure to yield a valid schedule, which is not our interest. Therefore, we only show values up to 2 to better illustrate results with valid schedules in more detail. The results are similar for RTC and RTC when almost 100 percent failure is observable in the tight deadline in Fig. 6b. As no plots is shown in this figure, it means the achieved cost by RCT and RTC are at least 2 times greater than defined budget.

The performance of BDHEFT contrasts with both RTC and RCT in the sense that its 100 percent failure is due to deadline violation which can be inferred by looking at 6b. When α is four, all cost ratios are placed in the valid

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.



(a) Time Efficiency



(b) Cost Efficiency

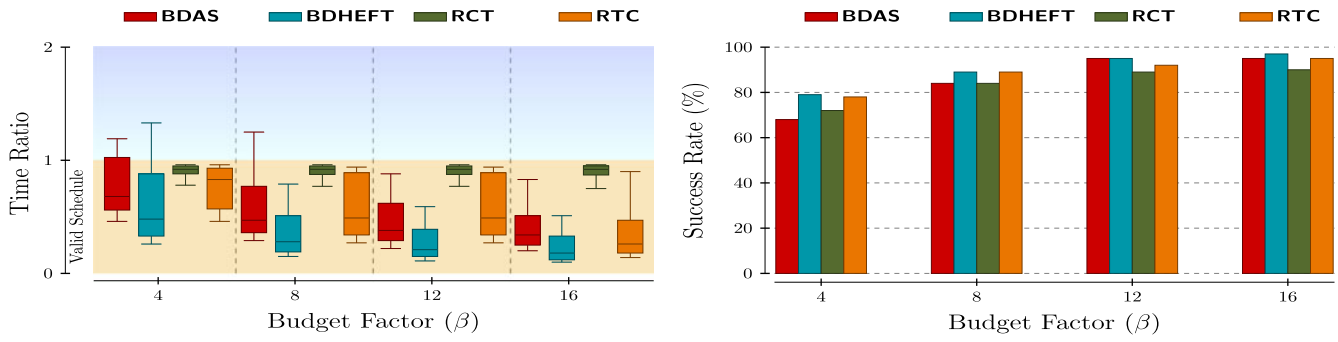
Fig. 4. LIGO.

schedule. However, the corresponding success rate report almost 100 percent failure. Therefore, it can be inferred that failures happened due to exceeding deadlines.

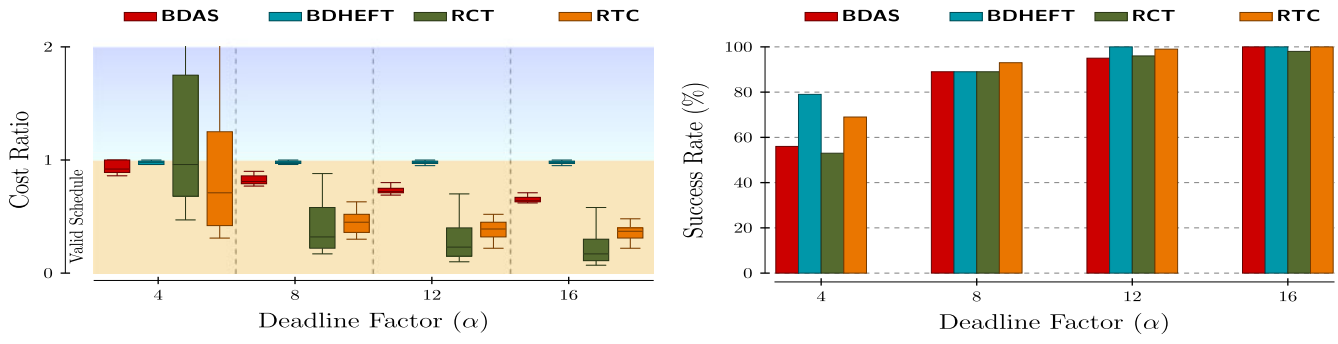
7.6 Total Success Rate

Table 4 shows the total Success Rate of each algorithm considering all values for budget factor, β, and deadline factor, α. For each workflow, we test 400 different states which is combination of 20 different budget factor and 20 different deadilne factor. We have considered 100 different workflows that leads to 40000 different test cases for each scientific workflow.

A low success rate indicates that the algorithm cannot find a schedule that meets the budget and deadline. The



(a) Time Efficiency



(b) Cost Efficiency

Fig. 5. MONTAGE.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

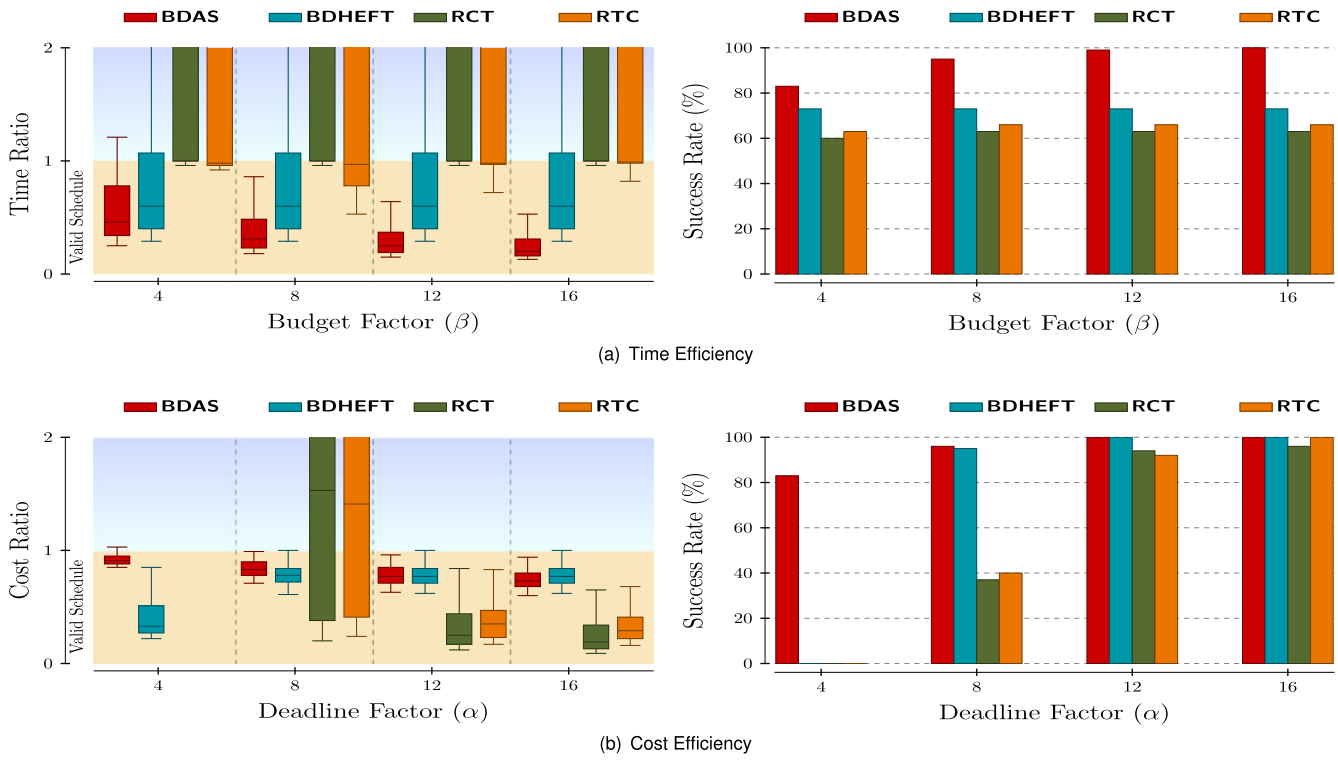


Fig. 6. SIPHT.

best overall performer is BDAS which exhibits a success of above 84 percent in all datasets with the best performance of 96 percent in Epigenomics. The highest failure and worst performance occurs in BDHEFT algorithm that fails around 60 percent of different test cases in EPIGENOMICS and LIGO.

Both RCT and RTC achieve similar performance with the highest success rates of 82 and 87 percent, both gained in MONTAGE, respectively. It is also worth noticing that MONTAGE workflow is the most successful application to be scheduled with all algorithms, based on reported success rate.

In general, our approach is more consistent and it is much more likely to generate acceptable schedules under defined constraints.

7.7 Summary of Scheduling Algorithms Performance

- In general, we can say that our algorithm utilize budget while finding a schedule without considerable failures.

TABLE 4
Total Success Rate for Five Different Scientific Workflows

	BDAS	BDHEFT	RCT	RTC
CYBERSHAKE	88%	84%	72%	64%
EPIGENOMICS	96%	38%	72%	71%
LIGO	86%	44%	69%	78%
MONTAGE	84%	90%	82%	87%
SIPHT	93%	72%	59%	62%
Mean	89.4%	65.6%	70.8%	72.4%

The mean Total Success Rate for BDAS is between 17.0–23.8 percent higher than other algorithms.

- The BDAS outperformed all algorithms in terms of success rate in specified tight budget/deadline factors.
- In contrast to BDAS, the performance analysis shows the both RCT and RTC utilize deadline while trying to find a cheaper schedule. In tight deadlines, most of the failures in RCT and RTC happen due to violating the budget constraint. This is the case in Figs. 2b, 3b and 6b.
- Considering all results, we can see that BDHEFT performs worse than other algorithms. It not only suffers from a large number of failures and constraint violation, but we also see the wide range on its cost and time ratios.
- Most of the failures in BDHEFT is due to violating deadline. for example, in SIPHT, when deadline factor is 4, the cost ratio obtained by BDHEFT is placed in the valid schedule. However, it achieves almost 100 percent failure due to violating the prescribed deadline. The same pattern is observable in the EPIGENOMICS workflow.

7.8 Sensitivity Analysis

The design of a scheduling strategy depends on what objectives are more important to users. In order to check how sensitive our algorithm is, in terms of assigning priority to budget and deadline, a sensitivity analysis is performed in this paper. Sensitivity analysis may be carried out to evaluate the stability and robustness of scheduling algorithms allowing users consider the preferred choice.

To achieve customizable services based on user requirements, we consider a weight parameter in our algorithm. With this mechanism, the sensitivity experiment analyzes how different weights on cost and time affect the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

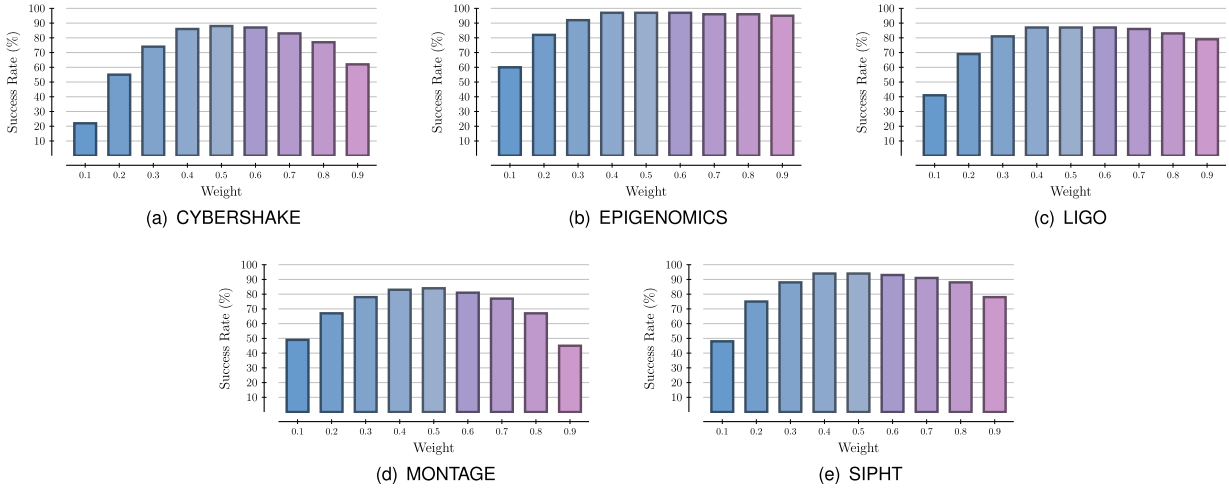


Fig. 7. Sensitivity Analysis for five different data set.

performance of the BDAS algorithm. The parameter ω takes values between 0 and 1, the smaller the value the larger the weight is on time while values close to 1 corresponds to assigning larger weight on cost. The weights modifies the CTTF in 20 as follows:

$$\mathrm{CTTF}_{t_i}^{p_j} = (\omega) * \mathrm{Cost}_{t_i}^{p_j} + (1-\omega) * \mathrm{Time}_{t_i}^{p_j}. \tag{28}$$

The results in Fig. 7 are shown up to their maximum possible number of success rates in total, no matter what budget factor and deadline factor are. The results of this set of experiments suggest that the structure and type of workflows can significantly impact the user priorities in scheduling workflows. Our analysis provides insights to the performance of scheduling algorithms with data intensive versus compute intensive workflows, of which the most important ones are as:

- Data Intensive: Cybershake, LIGO and MONTAGE, which are three data intensive workflows deal with large amount of data in the range of megabytes to petabytes and spend most of the running time performing I/O activities. As reported in [42], a MONTAGE workflow with 10000 tasks, reads 146 GB of input data, needs 4.93 CPU hours, and writes 49 GB of output data. In these data intensive applications, data transfer time between resources usually take a significant portion of the execution time. Therefore, reducing data movement in data intensive workflows is most effective in reducing costs. Assigning low priority to Cost in the trade-off equation introduced in 20 leads to higher number of failures. When the weight parameter, ω, is set to 0.1, 80 percent of experiments in Cybershake fail to meet the defined constraints while LIGO and MONTAGE workflows experienced of 60 and 55 percent failures, respectively.

  To have a better insights, we look to the Time and Cost equations given earlier in 18 and 19, respectively. The higher the value of Time the sooner an instance finishes its execution, which is probably an expensive instance. Because the expression in 28 has a higher weight on Time, more expensive resource are selected. This observation is confirmed by the results whereby all the failures in data intensive workflows were caused by exceeding the available budget.

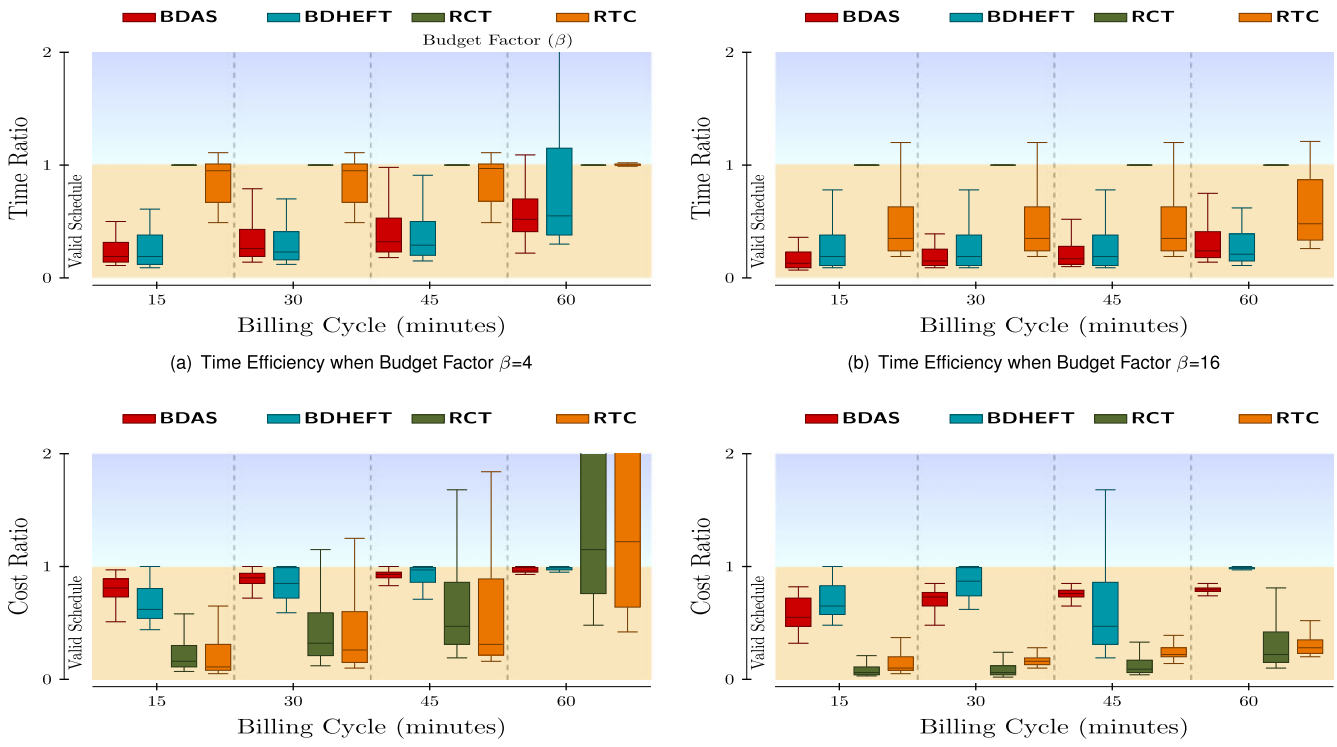
  Another interesting observation in data intensive applications is when a high weightage is assigned to Cost (i.e., ω = 0.9). The highest failure (60 percent) occurs with MONTAGE while CYBERSHAKE experienced 40 percent failures. This result is attributed to the interaction between Time and Cost in this scenario. A high value for Cost in 19 means that a task is executed during paid intervals that have no cost. Thus, the selection of the most suitable instance relies heavily on the availability of instances with zero (or very low) cost. However, such instances may fail to meet a task's sub-deadline leading to a failure due to exceeding the deadline.
- Compute Intensive: Large scale applications mainly consist of complex tasks requiring high performance computing instances are called compute intensive workflows. In these applications, most of the task's execution time utilizes computation rather than I/O resources. The EPIGENOMICS workflow is considered as CPU intensive application because it spends 99 percent of its execution utilizing CPU, and performs comparatively low I/O compared to Montage [42]. Higher priority given to time rather than cost causes 50 percent failure in SIPHT and 40 percent failure in EPIGENOMICS. This is a result of more expensive resources selected thus exceeding available budget. The experimental results in EPIGENOMICS show that the likelihood of meeting scheduling constraints increases by increasing the value ω. In Fig. 7, the general trend suggests that assigning higher priority to Cost yields fewer failures.

In summary, the results of the sensitivity analysis suggest that the different priorities have different impact in finding a valid schedule for each workflow type.

7.9 Decreasing Billing Cycle

In this section we investigate the Time Ratio and Cost Ratio performance of the four scheduling algorithms across different billing cycle (from 15 minutes to 1 hour with 15 minute increments). For the Time Ratio evaluations, we use a budget

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.



(a) Time Efficiency when Budget Factor β=4

(b) Time Efficiency when Budget Factor β=16

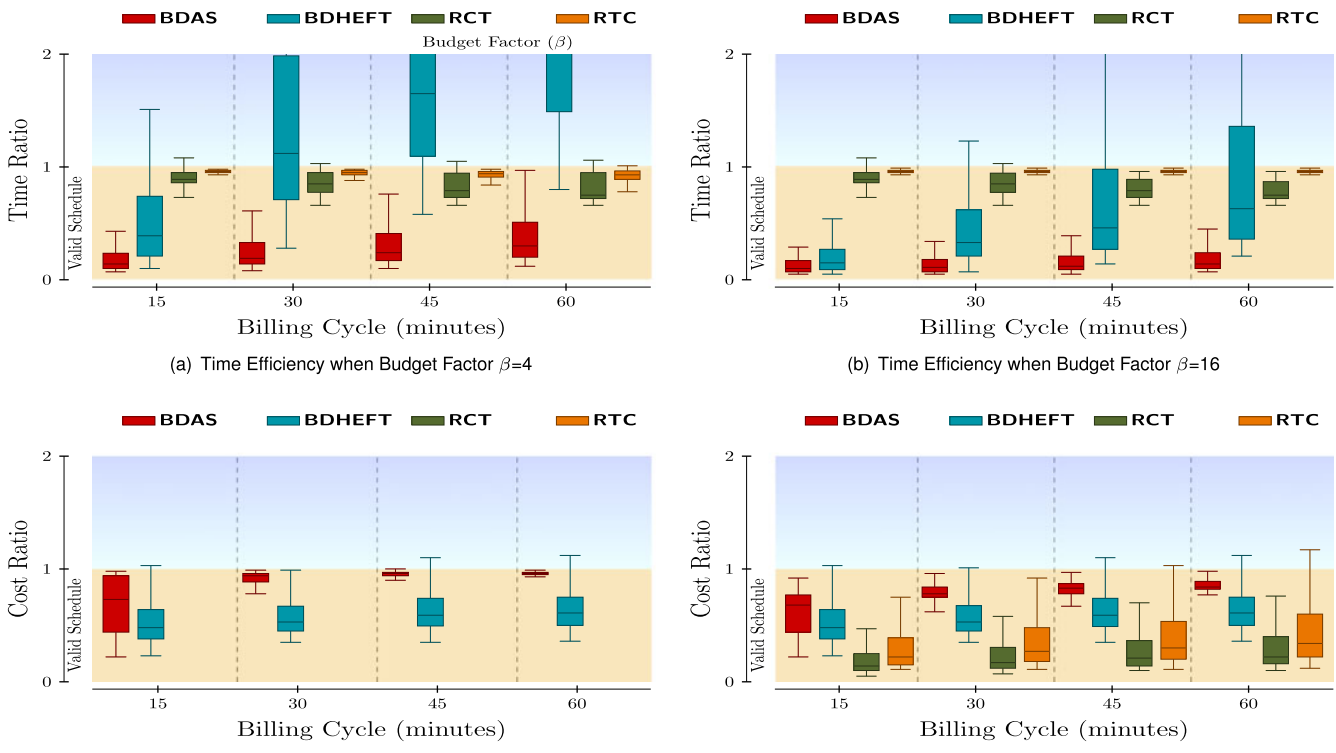
(c) Cost Efficiency when Deadline Factor α=4

(d) Cost Efficiency when Deadline Factor α=16

Fig. 8. CYBERSHAKE.

factor of $\beta = 4$ and $\beta = 16$, while Cost Ratio evaluations are performed with deadline factors fixed at $\alpha = 4$ and $\alpha = 16$.

Figs. 8, 9, 10, 11, 12 show the Time Ratio and Cost Ratio performance for CYBERSHAKE, LIGO, MONTAGE and SIPHT respectively. For BDAS and BDHEFT, the medians for Time Ratio marginally decrease with decreasing billing cycle while the range for Time Ratio also shrinks with decreasing billing cycle (from 60 minutes to 15 minutes). This observation for BDAS Time ratio holds across all four workflows for both budget factors $\beta = 4$ and $\beta = 16$. By



(a) Time Efficiency when Budget Factor β=4

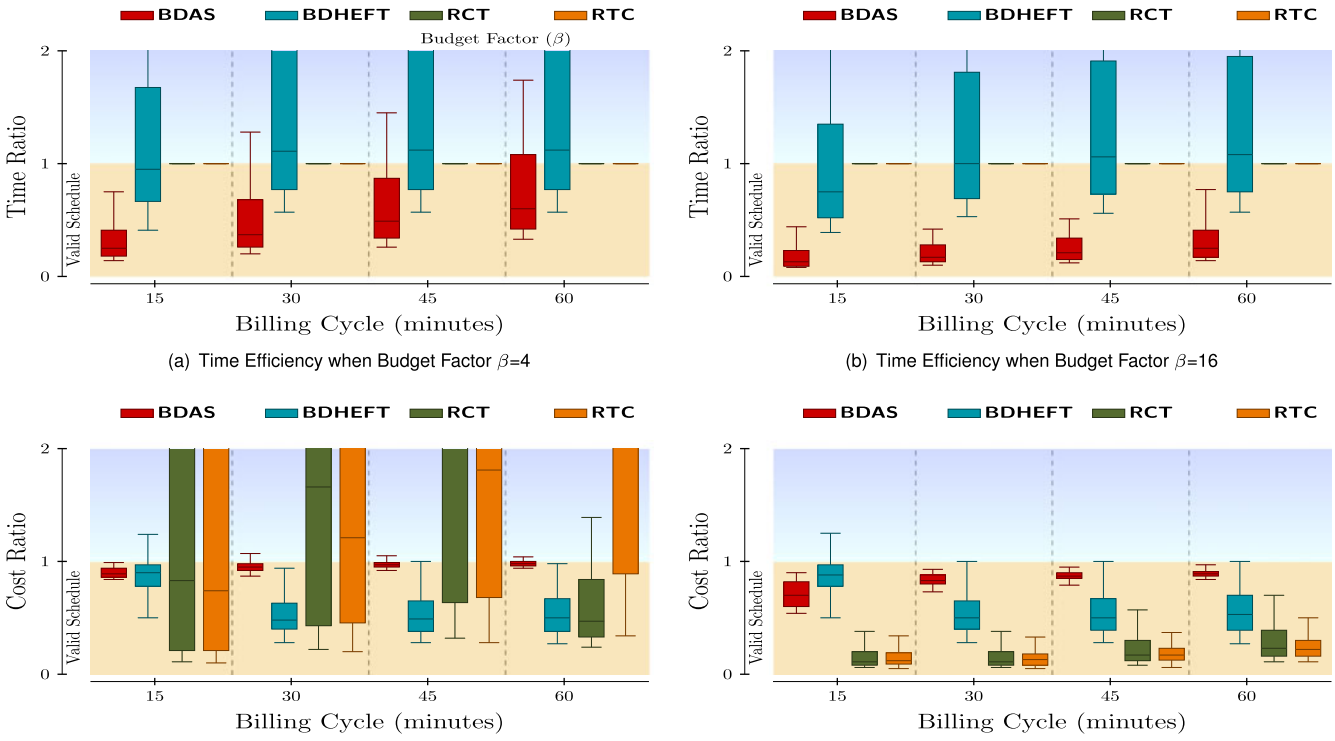
(b) Time Efficiency when Budget Factor β=16

(c) Cost Efficiency when Deadline Factor α=4

(d) Cost Efficiency when Deadline Factor α=16

Fig. 9. EPIGENOMICS.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.



(a) Time Efficiency when Budget Factor β=4

(b) Time Efficiency when Budget Factor β=16

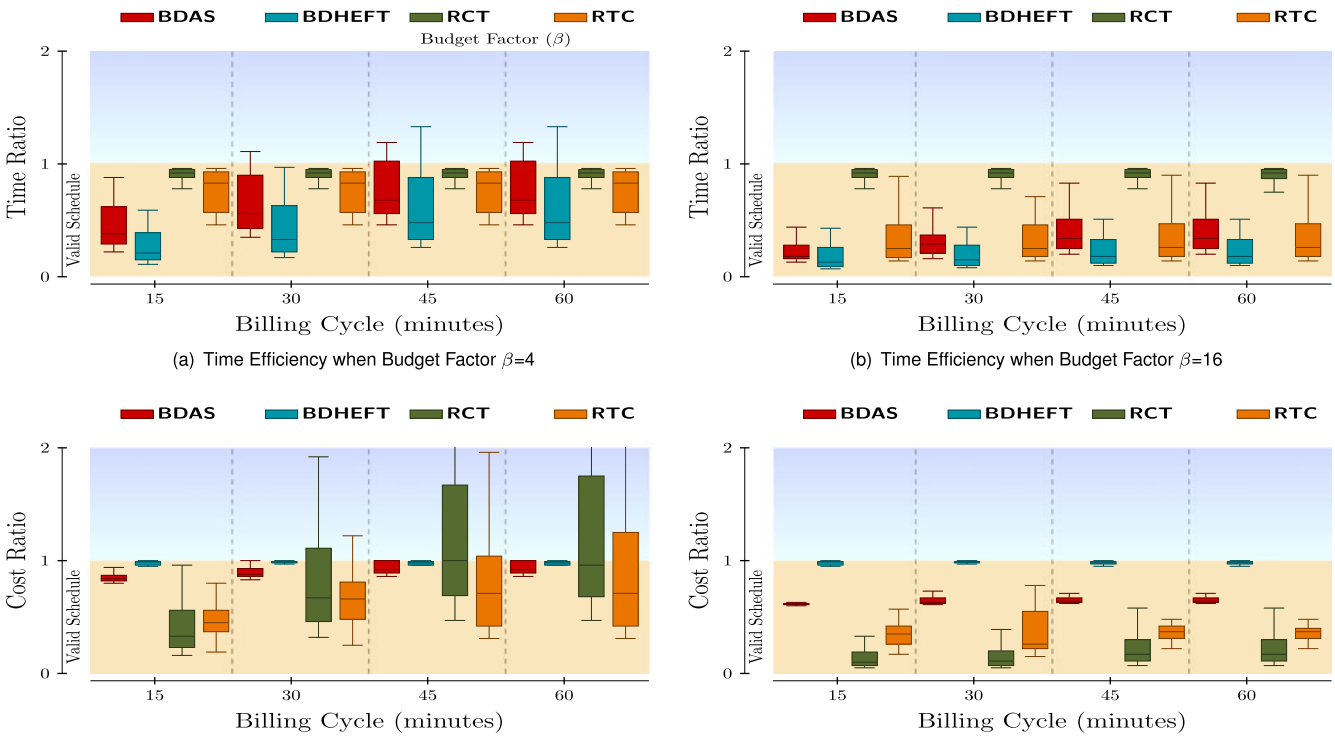
(c) Cost Efficiency when Deadline Factor α=4

(d) Cost Efficiency when Deadline Factor α=16

Fig. 10. LIGO.

decreasing the billing cycle, the number of idle time slots between scheduled tasks are decreased leaving more budget available for launching new VMs, thus resulting in a reduction of Time Ratio.

The Cost Ratio medians for BDAS and BDHEFT also decreases with decreasing billing cycle, however, the range for Cost Ratio slightly increases (converse of Time Ratio) suggesting that shorter billing cycles introduce more cost volatility. A larger range for Cost Ratio is observed in the CYBERSHAKE Fig. 8 and EPIGENOMICS Fig. 9, and to a lesser extent in LIGO Fig. 10, MONTAGE Fig. 11 and SIPHT Fig. 12. A shorter billing cycle means more granular updates



(a) Time Efficiency when Budget Factor β=4

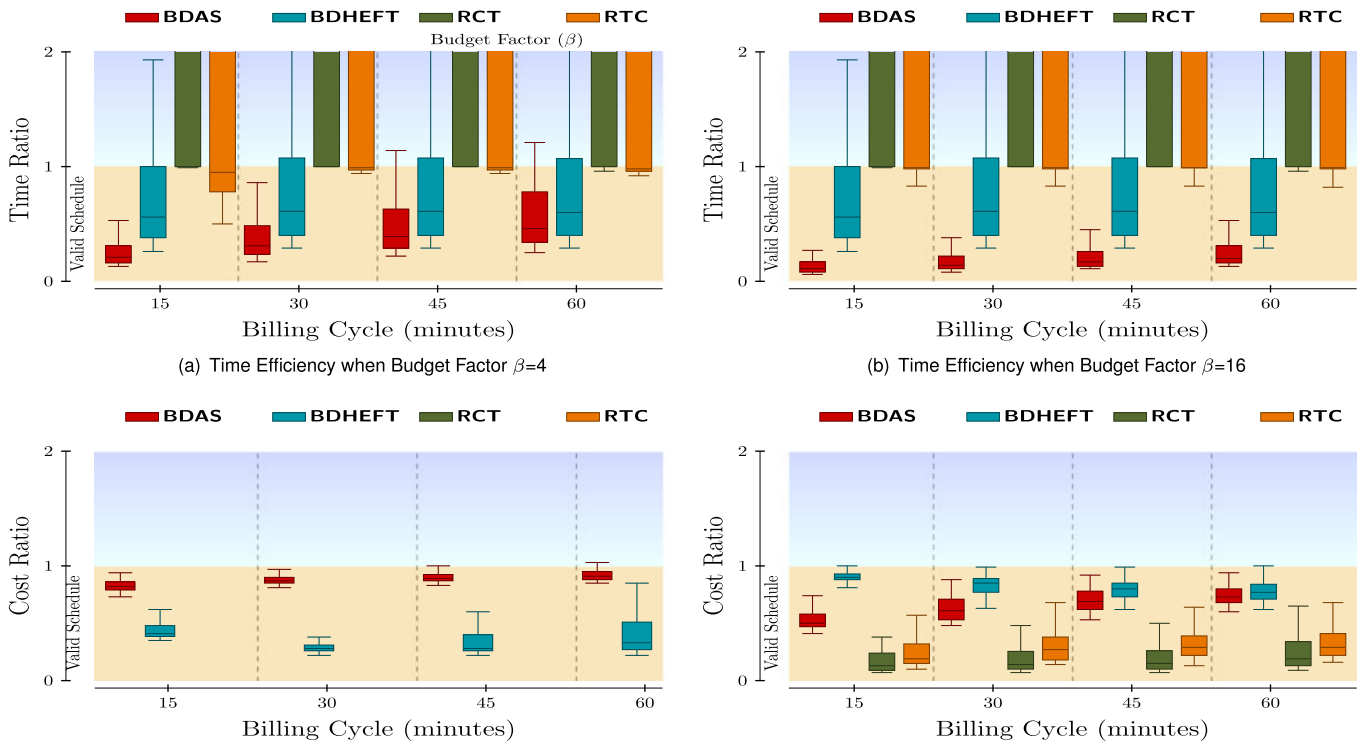
(b) Time Efficiency when Budget Factor β=16

(c) Cost Efficiency when Deadline Factor α=4

(d) Cost Efficiency when Deadline Factor α=16

Fig. 11. MONTAGE.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.



(a) Time Efficiency when Budget Factor β=4

(b) Time Efficiency when Budget Factor β=16

(c) Cost Efficiency when Deadline Factor α=4

(d) Cost Efficiency when Deadline Factor α=16

Fig. 12. SIPHT.

on incurred cost resulting in fewer idle instances, this helps reduce cost (as seen in the median trend). However, the more granular updates also increase the Cost ratio variance as observed in Figs. 8, 9, 10, 11, 12. Both RTC and RCT do not show significant variation in Time Ratio and Cost Ratio performance with decreasing billing cycle.

Overall, the different billing cycles have marginal influence of the Cost Ratio and Time Ratio; this finding concurs with another research by Rodrigues and Buyya [39].

8 Conclusion

In this paper we present a novel algorithm to address the problem of scientific workflow scheduling in dynamically provisioned commercial cloud environments. Our approach focuses on addressing the unique characteristics of workflow execution on cloud platforms, such as on-demand provisioning and instance heterogeneity, while simultaneously meeting budget and deadline constraints.

We evaluated the BDAS algorithm and compared it with three previously published algorithms (BDHEFT, RCT, RTC), using a range of metrics. In terms of success rate, the best overall performer is the BDAS algorithm which exhibits a success of above 84 percent in all datasets while the worst performance occurs in BDHEFT algorithm that fails around 60 percent of different test cases in EPIGENOMICS and LIGO. We also investigated the sensitivity of our algorithm on different workflow types over a range priorities for budget and deadline. Increasing budget on compute intensive workflows tends to achieve higher success rate, which implies that budget is more important than deadline for such applications.

A primary advantage of BDAS (and by extension heuristic algorithms), is its low overhead, or computational tractability. The scheduling performance achieved by the BDAS algorithm is promising, especially in light of the time required to compute a schedule. In future work, we intend to explore the use of the BDAS heuristic algorithm in near real-time, multiple workflow and dynamic scheduling situations.

References

[1] J. Towns, T. Cockerill, M. Dahan, I. Foster, K. Gaither, A. Grimshaw, V. Hazlewood, S. Lathrop, D. Lifka, G. D. Peterson, R. Roskies, J. R. Scott, and N. Wilkins-Diehr , "Xsede: Accelerating scientific discovery," *Comput. Sci. Eng.*, vol. 16, no. 5, pp. 62–74, Sep. 2014.

[2] B. P. Abbott, R. Abbott, et al., "Observation of gravitational waves from a binary black hole merger," *Phys. Rev. Lett.*, vol. 116, pp. 61–102, Feb. 2016.

[3] V. Mayer-Schonberger and K. Cukier, *Big Data: A Revolution That Will Transform How We Live, Work, and Think*. Boston, MA, USA: Houghton Mifflin, 2013.

[4] C. Vecchiola, S. Pandey, and R. Buyya, "High-performance cloud computing: A view of scientific applications," in *Proc. 10th Int. Symp. Pervasive Syst. Algorithms Netw.*, Dec. 2009, pp. 4–16.

[5] C. Evangelinos and C. N. Hill, "Cloud computing for parallel scientific hpc applications: Feasibility of running coupled atmosphere-ocean climate models on amazons ec2," in *Proc. 1st Workshop Cloud Comput. Appl.*, 2008, pp. 2–34.

[6] K. R. Jackson, L. Ramakrishnan, K. Muriki, S. Canon, S. Cholia, J. Shalf, H. J. Wasserman, and N. J. Wright, "Performance analysis of high performance computing applications on the amazon web services cloud," in *Proc. 2nd IEEE Int. Conf Cloud Comput. Technol. Sci.*, 2010, pp. 159–168.

[7] S. C. Park and S. Y. Ryoo, "An empirical investigation of end-users switching toward cloud computing: A two factor theory perspective," *Comput. Human Behavior*, vol. 29, no. 1, pp. 160–170, 2013.

[8] I. T. Foster and R. K. Madduri, "Science as a service: How on-demand computing can accelerate discovery," in *Proc. 4th ACM Workshop Sci. Cloud Comput.*, 2013, pp. 1–2.

[9] I. Foster, "Service-oriented science: scaling the application and impact of eresearch (abstract)," in *Proc. 1st Int. Conf. e-Sci. Grid Comput.*, Jul. 2005, pp. 1–2.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.

[10] R. Chard, K. Chard, K. Bubendorfer, L. Lacinski, R. Madduri, and I. Foster, "Cost-aware cloud provisioning," in *Proc. IEEE 11th Int. Conf. e-Sci.*, Aug. 2015, pp. 971–974.

[11] R. Chard, K. Chard, K. Bubendorfer, L. Lacinski, R. Madduri, and I. Foster, "Cost-aware elastic cloud provisioning for scientific workloads," in *Proc. 8th IEEE Int. Conf. Cloud Comput.*, Jun. 2015, pp. 136–144.

[12] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw.: Practice Exp.*, vol. 41, no. 1, pp. 23–50, 2011.

[13] I. J. Taylor, E. Deelman, D. B. Gannon, and M. Shields, *Workflows for e-Science: Scientific Workflows for Grids*. New York, NW, USA: Springer Publishing Company, Incorporated, 2014.

[14] S. M. Park and M. Humphrey, "Predictable high-performance computing using feedback control and admission control," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 3, pp. 396–411, Mar. 2011.

[15] X. Liu, J. Chen, and Y. Yang, *Temporal QoS Management in Scientific Cloud Workflow Systems*. Amsterdam, The Netherlands: Elsevier, 2012.

[16] J. Ullman, "Np-complete scheduling problems," *J. Comput. Syst. Sci.*, vol. 10, no. 3, pp. 384–393, 1975.

[17] F. Wu, Q. Wu, and Y. Tan, "Workflow scheduling in cloud: A survey," *J. Supercomput.*, vol. 71, pp. 1–46, 2015.

[18] R. Calheiros and R. Buyya, "Meeting deadlines of scientific workflows in public clouds with tasks replication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 7, pp. 1787–1796, Jul. 2014.

[19] R. Sakellariou, H. Zhao, E. Tsiakkouri, and M. D. Dikaiakos, "Scheduling workflows with budget constraints," in *Integrated Research in Grid Computing*, S. Gorlatch and M. Danelutto, Eds., CoreGrid series. New York, NY, USA: Springer-Verlag, 2007, pp. 189–202.

[20] K. Chard, K. Bubendorfer, and P. Komisarczuk, "High occupancy resource allocation for grid and cloud systems, a study with drive," in *Proc. 19th ACM Int. Symp. High Perform. Distrib. Comput.*, 2010, pp. 73–84.

[21] R. Chard, K. Chard, K. Bubendorfer, L. Lacinski, R. Madduri, and I. Foster, "Cost-aware cloud provisioning," in *Proc. IEEE 11th Int. Conf. e-Sci.*, Aug. 2015, pp. 136–144.

[22] J. Yu, M. Kirley, and R. Buyya, "Multi-objective planning for workflow execution on grids," in *Proc. 8th IEEE/ACM Int. Conf. Grid Comput.*, 2007, pp. 10–17.

[23] J. Yu and R. Buyya, "Scheduling scientific workflow applications with deadline and budget constraints using genetic algorithms," *Sci. Program.*, vol. 14, no. 3/4, pp. 217–230, 2006.

[24] W.-N. Chen and J. Zhang, "An ant colony optimization approach to a grid workflow scheduling problem with various qos requirements," *IEEE Trans. Syst. Man Cybern. Part C: Appl. Rev.*, vol. 39, no. 1, pp. 29–43, Jan. 2009.

[25] A. Verma and S. Kaushal, "Bi-criteria priority based particle swarm optimization workflow scheduling algorithm for cloud," in *Proc. Recent Adv. Eng. Comput. Sci.*, Mar. 2014, pp. 1–6.

[26] K. Liu, H. Jin, J. Chen, X. Liu, D. Yuan, and Y. Yang, "A compromised-time-cost scheduling algorithm in swindew-c for instance-intensive cost-constrained workflows on a cloud computing platform," *Int. J. High Perform. Comput. Appl.*, vol. 24, no. 4, pp. 445–456, Nov. 2010.

[27] W. Zheng and R. Sakellariou, "Budget-deadline constrained workflow planning for admission control," *J. Grid Comput.*, vol. 11, no. 4, pp. 633–651, 2013.

[28] A. Verma and S. Kaushal, "Cost-time efficient scheduling plan for executing workflows in the cloud," *J. Grid Comput.*, vol. 13, no. 4, pp. 495–506, 2015.

[29] D. Poola, S. Garg, R. Buyya, Y. Yang, and K. Ramamohanarao, "Robust scheduling of scientific workflows with deadline and budget constraints in clouds," in *Proc. IEEE 28th Int. Conf. Adv. Inf. Netw. Appl.*, May 2014, pp. 858–865.

[30] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Gener. Comput. Syst.*, vol. 29, no. 1, pp. 158–169, 2013, including Special section: AIRCC-NetCoM 2009 and Special section: Clouds and Service-Oriented Architectures.

[31] M. Malawski, G. Juve, E. Deelman, and J. Nabrzyski, "Algorithms for cost- and deadline-constrained provisioning for scientific workflow ensembles in iaas clouds," *Future Gener. Comput. Syst.*, vol. 48, pp. 1–18, 2015.

[32] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M. H. Su, and K. Vahi, "Characterization of scientific workflows," in *Proc. 3rd Workshop Workflows Support Large-Scale Sci.*, Nov. 2008, pp. 1–10.

[33] Y. Yuan, X. Li, Q. Wang, and Y. Zhang, "Bottom level based heuristic for workflow scheduling in grids," *Chinese J. Comput.-Chinese Ed.*, vol. 31, no. 2, 2008, Art. no. 282.

[34] J. Yu, R. Buyya, and C. K. Tham, "Cost-based scheduling of scientific workflow applications on utility grids," in *Proc. 1st Int. Conf. e-Sci. Grid Comput.*, Jul. 2005, pp. 8–147.

[35] V. Arabnejad, K. Bubendorfer, and B. Ng, "Budget distribution strategies for scientific workflow scheduling in commercial clouds," in *Proc. IEEE 12th Int. Conf. e-Sci.*, Oct. 2016, pp. 137–146.

[36] V. Arabnejad, K. Bubendorfer, and B. Ng, "Deadline distribution strategies for scientific workflow scheduling in commercial clouds," in *Proc. 9th Int. Conf. Utility Cloud Comput.*, 2016, pp. 70–78.

[37] M. R. Palankar, A. Iamnitchi, M. Ripeanu, and S. Garfinkel, "Amazon S3 for science grids: a viable solution?" in *Proc. Int. Workshop Data-Aware Distrib. Comput.*, 2008, pp. 55–64.

[38] S. Ostermann, A. Iosup, N. Yigitbasi, R. Prodan, T. Fahringer, and D. Epema, "A performance analysis of ec2 cloud computing services for scientific computing," in *Cloud Computing*. Berlin, Germany: Springer, 2010, vol. 34, pp. 115–131.

[39] M. A. Rodriguez and R. Buyya, "Budget-driven scheduling of scientific workflows in IaaS clouds with fine-grained billing periods," *ACM Trans. Auton. Adapt. Syst.*, vol. 12, no. 2, pp. 5:1–5:22, May 2017.

[40] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioningand scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Apr. 2014.

[41] M. Mao and M. Humphrey, "A performance study on the vm startup time in the cloud," in *Proc. IEEE 5th Int. Conf. Cloud Comput.*, 2012, pp. 423–430.

[42] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, "Characterizing and profiling scientific workflows," *Future Gener. Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.

Vahid Arabnejad is working toward the PhD degree in the School of Engineering and Computer Science, Victoria University of Wellington (VUW), New Zealand. His main research interests include parallel and distributed computing. His research focuses on resource management and scheduling techniques in clouds.

Kris Bubendorfer is an associate professor in engineering with Victoria University of Wellington. His research interests include cloud related areas of distributed computing, including, services and service oriented architectures, cloud computing markets, resource allocation, cyber security, high performance computing, scientific computation, eResearch and the associated topics of social computing and reputation systems.

Bryan Ng working toward the network engineering and statistics at University Malaya. He is a lecturer with Victoria University. His research interests include mathematical modelling and performance analysis of networks and protocols. Prior to joining Victoria, he worked with Orange Labs, INRIA and AC Nielsen consulting.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:54:14 UTC from IEEE Xplore. Restrictions apply.